دانشگاه تهران
دانشکدگان فنی

برنامه‌سازی پیشرفته

تمرین

تمرین اول

۰۱

سفر با کشتی پژوهشی

به کاوشگر جاوا خوش آمدید!

کشتی پژوهشی «کاوشگر جاوا» در میانهٔ یک سفر علمی، سامانه‌های هدایت، ارتباط و پایش خود را می‌آزماید. شما به تیم برنامه‌نویسی کشتی پیوسته‌اید تا در شش مأموریت، از هدایت کشتی و عبور از میدان شناورها تا ارتباط با ماهواره، بررسی محمولهٔ زیستی و عیب‌یابی سامانهٔ کنترل را پیش ببرید.

هر مأموریت مسئله‌ای مستقل است، اما همه بخشی از یک سفرند: کشتی باید با کمک راه‌حل‌های شما با اطمینان به مقصد برسد. گاهی باید برنامه‌ای بنویسید و گاهی رفتار برنامه را تحلیل کنید. جزئیات مهم‌اند؛ گاهی یک شرط کوچک است که مسیر را عوض می‌کند.

پیش از آغاز مأموریت‌ها

- تمام برنامه‌ها باید با Java نوشته شوند.
- فقط از مفاهیمی استفاده کنید که تا این بخش از درس تدریس شده‌اند. به‌کارگیری `Array`، `ArrayList`، بازگشت یا هر مبحث تدریس‌نشده‌ای مجاز نیست، مگر آنکه صورت سؤال صریحاً آن را مجاز یا لازم بداند.
- در سؤالات گرافیکی از کتابخانهٔ Stanford ACM Graphics استفاده کنید.
- پاسخ بخش‌های تحلیلی را با **استدلال کوتاه** پشتیبانی کنید؛ نوشتن `True`، `False` یا خروجی نهایی به‌تنهایی کافی نیست.
- نام‌های معنادار برای متغیرها انتخاب کنید و کد را خوانا نگه دارید. برنامه باید برای ورودی‌های معتبر گوناگون کار کند، نه فقط نمونهٔ صورت سؤال.
- پیش از تحویل، برنامه‌ها را اجرا و بررسی کنید و از کامل‌بودن فایل‌های خواسته‌شده مطمئن شوید.

مأموریت ۱

سیستم هدایت کشتی

۱.۵ نمره

کشتی پژوهشی «کاوشگر جاوا» برای آزمایش سیستم هدایت خودکار وارد یک منطقهٔ کنترل‌شده به ابعاد 600×400 پیکسل شده است.
کشتی را با یک `GOval` به قطر 40 پیکسل نمایش دهید و آن را در ابتدای برنامه در مرکز صفحه قرار دهید.
سرعت اولیه:

```
double dx = 3;
double dy = 2;
```

بخش عملی (۱ نمره)

برنامه‌ای بنویسید که کشتی به صورت پیوسته حرکت کند.

- هنگام برخورد با دیوارهٔ چپ یا راست، جهت افقی حرکت تغییر کند.
- هنگام برخورد با دیوارهٔ بالا یا پایین، جهت عمودی حرکت تغییر کند.
- اندازهٔ واقعی `GOval` هنگام تشخیص برخورد در نظر گرفته شود.
- پس از 10 برخورد، حرکت متوقف شود و عبارت زیر روی صفحه نمایش داده شود:

```
MISSION COMPLETED
```

بخش تحلیلی (۰.۵ نمره)

چرا شرط زیر برای تشخیص برخورد با دیوارهٔ راست انتخاب مناسبی نیست؟

```
x == 600
```

شرط مناسب‌تری پیشنهاد کنید و دلیل خود را توضیح دهید.

مأموریت ۲

عبور از میدان شناورها

۱.۵ نمره

کاوشگر جاوا وارد منطقه‌ای شده که شناورهای هشداردهنده، محدودهٔ امن حرکت را مشخص می‌کنند.

بخش اول: رسم شناورها (۰.۵ نمره)

با استفاده از حلقه‌های تو در تو یک شبکهٔ 5×5 از شناورها رسم کنید.
هر شناور یک `GOval` با قطر 20 پیکسل است. فاصلهٔ بین مرکز دو شناور مجاور 60 پیکسل است و اولین شناور از مختصات زیر شروع می‌شود:

```
x = 50
y = 50
```

مختصات سایر شناورها باید توسط برنامه محاسبه شود.

بخش دوم: حرکت کشتی (۰.۷۵ نمره)

کشتی را با یک `GRect` به اندازهٔ 20×30 پیکسل در پایین صفحه نمایش دهید. سرعت افقی کشتی برابر است با:

```
double dx = 4;
```

کشتی باید بین دیوارهٔ چپ و راست حرکت کند و هر بار که به یکی از این دو دیواره رسید، جهت حرکت خود را تغییر دهد.
پس از 6 بار تغییر جهت، کشتی متوقف شود.

بخش تحلیلی (۰.۲۵ نمره)

کدام شرط برای تشخیص برخورد کشتی با سمت راست صفحه مناسب‌تر است؟ چرا؟

```
ship.getX() >= 600
```

یا:

```
ship.getX() + ship.getWidth() >= getWidth()
```

پاسخ خود را حداکثر در دو جمله توضیح دهید.

چالش اختیاری

بدون اضافه کردن حلقه‌ای جدید، کاری کنید شناورهایی که شمارهٔ سطر و ستون آن‌ها هر دو زوج است، بزرگ‌تر از سایر شناورها رسم شوند.

مأموریت ۳

ارتباط با ماهواره

۱.۵ نمره

کشتی برای ارسال اطلاعات علمی از یک ماهواره استفاده می‌کند. ماهواره در هر بازهٔ 10 ثانیه‌ای حداکثر 3 درخواست را قبول می‌کند. بازه‌ها به شکل زیر هستند:

$$[0, 10), \quad [10, 20), \quad [20, 30), \quad \dots$$

بخش عملی (۱ نمره)

زمان درخواست‌ها به ترتیب صعودی از ورودی دریافت می‌شود. برای هر درخواست:

- اگر کمتر از 3 درخواست در بازهٔ فعلی پذیرفته شده باشد، `ALLOWED` چاپ شود.
- در غیر این صورت `BLOCKED` چاپ شود.
- با ورود به یک بازهٔ جدید، شمارندهٔ درخواست‌ها از صفر شروع شود.

برای مثال:

ورودی

```
1
3
5
7
12
```

خروجی

```
ALLOWED
ALLOWED
ALLOWED
BLOCKED
ALLOWED
```

بخش تحلیلی (۰.۵ نمره)

برای تعیین شمارهٔ بازه از عبارت زیر استفاده شده است:

```
int window = timestamp / 10;
```

بدون اجرای برنامه، شمارهٔ `window` را برای مقادیر زیر مشخص کنید:

```
timestamp = 9
timestamp = 10
timestamp = 29
timestamp = 30
```

سپس کوتاه توضیح دهید چرا تقسیم صحیح برای این مسئله مناسب است.

مأموریت ۴
محمولهٔ زیستی
۱ نمره

دانشمندان کشتی موجودی را بررسی می‌کنند که جمعیت آن طبق الگوی زیر رشد می‌کند:
$$1, 1, 2, 3, 5, 8, 13, \ldots$$

بخش عملی (۰.۷۵ نمره)
عدد مثبت N را دریافت کرده و جملهٔ Nام دنبالهٔ فیبوناچی را با استفاده از یک حلقه محاسبه و چاپ کنید.

```
Input: 7
Output: 13
```

استفاده از Array و بازگشت مجاز نیست.

بخش تحلیلی (۰.۲۵ نمره)
آیا برای محاسبهٔ جملهٔ بعدی فیبوناچی لازم است تمام جمله‌های قبلی را نگهداری کنیم؟ اگر پاسخ منفی است، حداقل چه اطلاعاتی باید از مرحلهٔ قبل نگهداری شود؟

مأموریت ۵
باگ سیستم کنترل
۱ نمره

یکی از سیستم‌های کشتی رفتار غیرمنتظره‌ای دارد. کد زیر از سیستم استخراج شده است. **بدون اجرای کد پاسخ دهید.**

```
int x = 4;
int y = 2;

if (x % 2 == 0 || ++y > 5) {
    x += y;
}

System.out.println(x);
System.out.println(y);
```

الف (۰.۲۵ نمره)
مقادیر نهایی x و y را مشخص کنید.

ب (۰.۲۵ نمره)
آیا عبارت ++y اجرا می‌شود؟ دلیل خود را توضیح دهید.

ج (۰.۵ نمره)
اگر || با && جایگزین شود، آیا مقدار نهایی y تغییر می‌کند؟ مفهوم ارزیابی اتصال کوتاه (Short-Circuit Evaluation) را در توضیح خود به‌کار ببرید.

مأموریت ۶

مقصد در دیدرس است!

۱ نمره

کاوشگر جاوا به آخرین بخش سفر رسیده است. سه مقدار زیر را از ورودی دریافت کنید:

```
distance
speed
fuel
```

در هر مرحله:

- کشتی به اندازهٔ `speed` به مقصد نزدیک می‌شود.
- یک واحد سوخت مصرف می‌شود.

حرکت تا زمانی ادامه دارد که کشتی به مقصد برسد یا سوخت آن تمام شود.

بخش عملی (۰.۷۵ نمره)

برنامه را به‌گونه‌ای پیاده‌سازی کنید که اگر کشتی به مقصد رسید، چاپ شود:

```
ARRIVED
```

و اگر ابتدا سوخت تمام شد:

```
OUT OF FUEL
```

همچنین تعداد مراحل طی‌شده را چاپ کنید.

بخش تحلیلی (۰.۲۵ نمره)

برای این مسئله استفاده از `while` مناسب‌تر است یا `for`؟ پاسخ خود را در یک یا دو جمله توضیح دهید.

نحوهٔ تحویل تمرین

تمام فایل‌های مربوط به تمرین باید در یک فایل ZIP قرار گرفته و همان فایل ارسال شود. ساختار پیشنهادی فایل تحویلی:

```
StudentNumber_Name_HW1.zip
├── Answers.pdf
├── Mission1/
│   └── Mission1.java
├── Mission2/
│   └── Mission2.java
├── Mission3/
│   └── Mission3.java
├── Mission4/
│   └── Mission4.java
└── Mission6/
    └── Mission6.java
```

نام فایل نهایی: `StudentNumber_Name_HW1.zip`

فایل پاسخ‌ها

پاسخ تمام قسمت‌های تحلیلی و تئوری را در یک فایل Word یا Google Docs بنویسید و در پایان آن را به فرمت PDF تبدیل کنید. فایل PDF باید شامل موارد زیر باشد:

- نام و نام خانوادگی
- شمارهٔ دانشجویی
- شمارهٔ هر مأموریت
- پاسخ بخش‌های تحلیلی
- پاسخ سؤال 5 که نیاز به فایل Java جداگانه ندارد

نام پیشنهادی: `Answers.pdf`

فایل‌های کد

برای هر مأموریت عملی، فایل یا پروژهٔ Java مربوط به آن را نیز در پوشهٔ جداگانه قرار دهید. **کد باید قابل اجرا باشد.**

از قرار دادن کد صرفاً به صورت عکس یا تصویر صفحه خودداری کنید. در صورت استفاده از Stanford ACM Graphics، تمام فایل‌های لازم پروژه را به شکلی قرار دهید که کد قابل بررسی باشد.

چک‌لیست قبل از ارسال

- ☐ فایل PDF پاسخ‌ها داخل ZIP قرار دارد.
- ☐ تمام فایل‌های java مورد نیاز داخل ZIP هستند.
- ☐ برنامه‌ها قبل از تحویل اجرا و بررسی شده‌اند.
- ☐ فایل ZIP خراب نیست و باز می‌شود.
- ☐ نام فایل ZIP مطابق فرمت خواسته‌شده است.
- ☐ فایل‌های اضافی و غیرمرتبط داخل ZIP قرار نگرفته‌اند.
- ☐ پاسخ‌های تئوری فقط در کد یا توضیحات درون‌برنامه‌ای نوشته نشده‌اند و در PDF نیز وجود دارند.

پایان مأموریت

اگر به اینجا رسیدید، کاوشگر جاوا با موفقیت سفر خود را به پایان رسانده است. در این تمرین با بخش مهمی از ابزارهایی که تا اینجا یاد گرفته‌اید کار کردید: متغیرها • عبارت‌ها • شرط if/else • منطق بولی • حلقه‌ها • تقسیم صحیح • ارزیابی اتصال کوتاه • GRect • GOval • حلقه‌های تو در تو • پویانمایی

ارتباط با دستیاران آموزشی

در صورتی که دربارهٔ صورت سؤال‌ها، نحوهٔ پیاده‌سازی یا بخش‌های مختلف تمرین سؤالی داشتید، می‌توانید سؤال خود را از دستیاران آموزشی مربوطه در تلگرام بپرسید:

ReyhaneSamimi — @kamiRSH — @radmankhodaei — @ahmd859097

موفق باشید.